Deep Learning with Applications in Natural Language Processing

Lecture 3. Syntactic parsing: constituency and dependency

Mihaela Breabăn · Mădălina Răschip · Diana Trandabăț
Faculty of Computer Science, Alexandru Ioan Cuza University of Iași
Academic year 2026–2027

Today

- From word-level analysis to sentence structure
- Constituents and phrase structure
- Formal grammars and the Chomsky hierarchy
- Context-free grammars and probabilistic CFGs
- Chart parsing: CKY and Earley
- Dependency grammars and dependency parsing
- From syntax to meaning: semantic roles

Where Lecture 1 and Lecture 2 stopped

Lecture 1 tokens · lemmas · part-of-speech tags

Lecture 2 probabilities for word/token sequences

the	**flight**	**includes**	**a**	**meal**
DET	NOUN	VERB	DET	NOUN

Today: how words combine, which words form phrases, and how words depend on each other.

In classical constituency parsing, POS tags are the pre-terminals.
Modern neural parsers may predict POS jointly or work directly from contextual representations.

Syntactic parsing

Phrase structure (constituency)

[S [NP John]
  [VP kicks [NP Bill]]]

Dependency

kicks → John (nsubj)
kicks → Bill (obj)

PART 1

Constituency

Sentences are built from nested phrases.

Phrase structure grammar

- A sentence breaks into constituents (phrases), which break into smaller ones.
- Phrase types: NP (noun phrase), VP (verb phrase), PP (prepositional phrase), …
- Structures are recursive: an NP can contain another NP.
- [NP the old clever tall blue-eyed man]
- [NP [NP the man] [PP with [NP the hat]]]

How do we know if a group is a constituent?

Test	Example
It can stand alone	What did you find? — A puppy. (not “found a”)
A pronoun can replace it	I found a puppy → I found him.
It can be moved	A puppy was found by the child.
It can be clefted	It was a puppy that the child found.

A sentence is more than a string

I saw the lady with the binoculars

The same string, two different structures:

Who has the binoculars?

[I] saw the lady [with the binoculars]
→ I use the binoculars

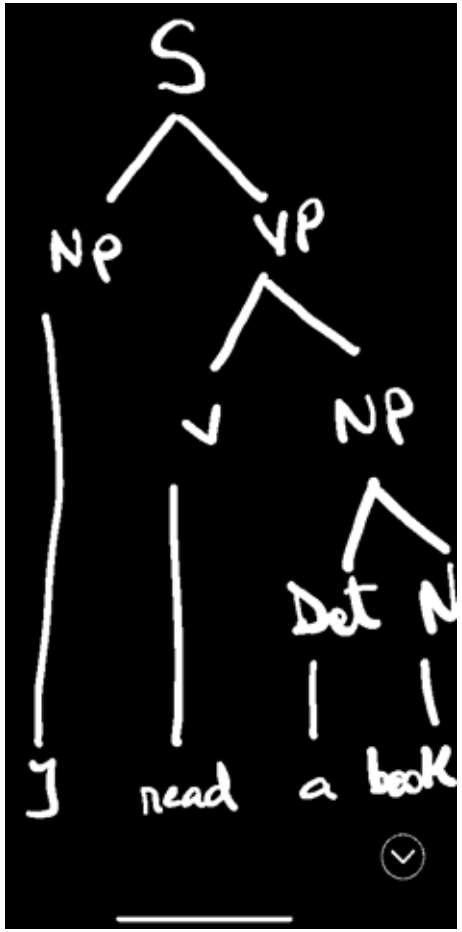
I saw [the lady with the binoculars]
→ the lady has the binoculars

Representing structure with trees

Bracket notation

```
[S [NP I]
   [VP [V read]
      [NP [Det the] [N book]]]]
```

Read as a tree



S has two children: NP and VP
VP has two children: V and NP
The words are the leaves

Structural ambiguity: a classic

PP inside the NP

[VP shot [NP an elephant
[PP in my pajamas]]]

The elephant wears the pajamas.

PP attached to the VP

[VP [VP shot an elephant]
[PP in my pajamas]]

I wear the pajamas.

PART 2

Formal grammars

How powerful must a grammar be to describe natural language?

The Chomsky hierarchy

Type	Grammar	Rule form	Automaton
0	unrestricted	$\alpha \to \beta$ (α non-empty)	Turing machine
1	context-sensitive	$\alpha A \beta \to \alpha \gamma \beta$	linear-bounded
2	context-free	$A \to \gamma$	push-down
3	regular	$A \to xB$ or $A \to x$	finite-state

Type 3 $\subset$ Type 2 $\subset$ Type 1 $\subset$ Type 0: each class strictly contains the next.

Regular vs context-free

Language	Smallest class	Natural-language analogue
$a^m b^n$	regular	adjectives before a noun
$a^n b^n$	context-free	nested subject–verb pairs
$a^n b^m c^n d^m$	context-sensitive	Swiss German (appendix)

Regular grammars cannot count matching pairs; context-free grammars can, through recursion.

Is English regular? Centre embedding

Sentence	Pattern
The cat likes tuna fish.	a b
The cat the dog chased likes tuna fish.	a a b b
The cat the dog the rat bit chased likes tuna fish.	a a a b b b

Each subject needs its own verb: the pattern is $a^n b^n$, which no regular grammar can generate.

PART 3

Context-free grammars

The workhorse of constituency parsing.

Context-free grammars

A CFG is a tuple (N, Σ, R, S):

N: non-terminals (S, NP, VP, Det, N)
Σ: terminals, the words
R: rules A → γ, one non-terminal on the left
S: the start symbol

```
S → NP VP
NP → Det N
Det → the
N → dog
```

Pre-terminals (Det, N) rewrite to words: they play the role of POS tags.

A derivation, step by step

```
S → NP VP                S ⇒ NP VP
NP → N                   ⇒ N VP
VP → V NP                ⇒ John VP
V → kicks                ⇒ John V NP
N → John | Bill          ⇒ John kicks NP
                         ⇒ John kicks N
                         ⇒ John kicks Bill
```

A different grammar, a different structure

Grammar A

S → NP VP
VP → V NP

[S [NP John]
  [VP kicks [NP Bill]]]

Grammar B

S → NP V NP

[S [NP John] kicks
  [NP Bill]]

Ambiguous grammars

```
NP → NP CONJ NP
NP → N
CONJ → and
N → John | Bill | Mary
```

John and Bill and Mary gets two structures:

[[John and Bill] and Mary]
[John and [Bill and Mary]]

A good grammar does not produce too many structures for the same sentence.

Evaluating a grammar

Question	Example of a problem
Undergenerates?	no PP rule: rejects “John kicks Bill in the park”
Overgenerates?	accepts “the dog *the”
Appropriate structures?	Grammar B above: no VP
Simple?	few rules, few special cases
How ambiguous?	structures per sentence

PART 4

Probabilistic CFGs

Choosing between parses with probabilities.

Probabilistic CFG (PCFG)

Lecture 2: the probability of a word sequence. Now: the probability of a parse.

- Each rule $A \rightarrow \beta$ has a probability $P(A \rightarrow \beta \mid A)$
- For every non-terminal, its rules' probabilities sum to 1
- Rule probabilities are estimated by MLE from a treebank, just like n-gram probabilities.

```
S → NP VP          0.8
S → Aux NP VP      0.1
S → VP             0.1

sum = 1.0
```

A small PCFG for English

Grammar rules

S → NP VP	.8
S → Aux NP VP	.1
S → VP	.1
NP → Pronoun	.2
NP → Proper-Noun	.2
NP → Det Nominal	.6
Nominal → Noun	.3
Nominal → Nominal Noun	.2
Nominal → Nominal PP	.5
VP → Verb	.2
VP → Verb NP	.5
VP → VP PP	.3
PP → Prep NP	1.0

Lexical rules (words)

Det → the | a | that | this
.6 .2 .1 .1

Noun → book | flight | meal | money
.1 .5 .2 .2

Verb → book | include | prefer
.5 .2 .3

Pronoun → I | he | she | me
.5 .1 .1 .3

Proper-Noun → Houston | NWA (.8, .2)
Aux → does 1.0
Prep → from | to | on | near | through
.25 .25 .1 .2 .2

For each left-hand side, the probabilities of its alternative expansions sum to 1.

Probability of a tree and of a sentence

PCFG rule choices are treated as independent given the left-hand side.

$$P(\text{tree}) = \prod \text{rule probabilities}$$

Choose the most probable tree to resolve ambiguity.
P(sentence) = sum of the probabilities of all its trees.

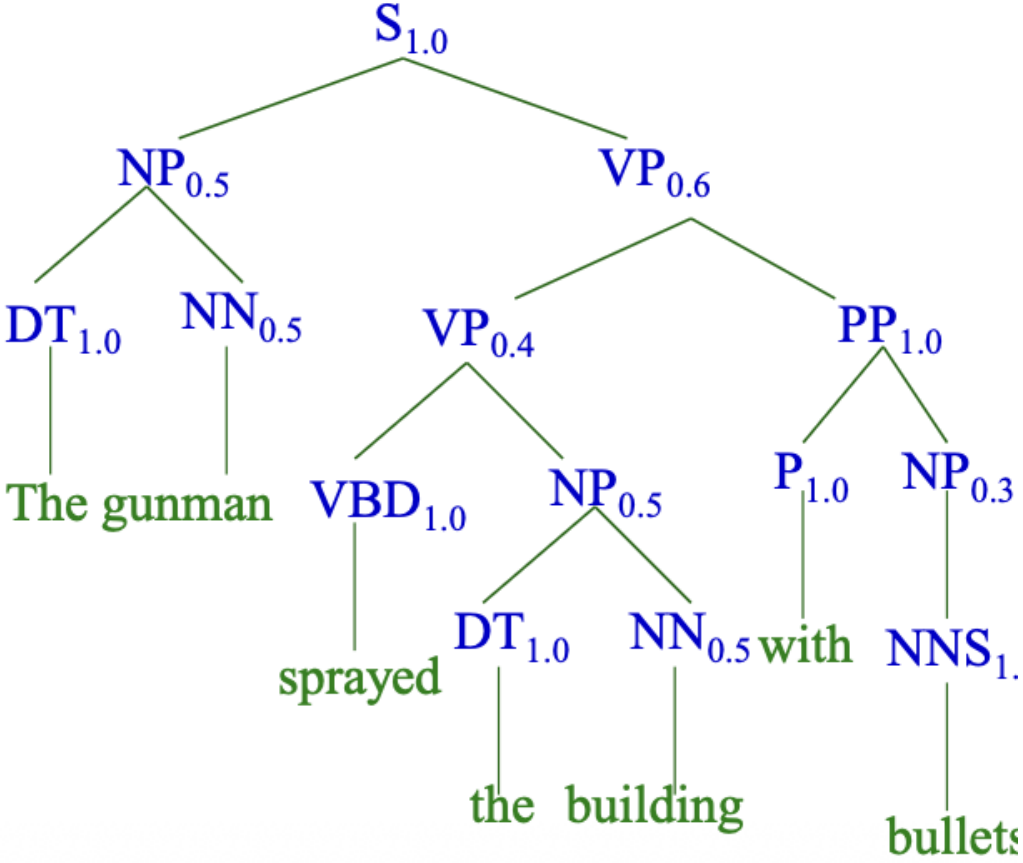
Example: the gunman

Rule	P	Rule	P
S → NP VP	1.0	DT → the	1.0
NP → DT NN	0.5	NN → gunman	0.5
NP → NNS	0.3	NN → building	0.5
NP → NP PP	0.2	VBD → sprayed	1.0
PP → P NP	1.0	NNS → bullets	1.0
VP → VP PP	0.6	P → with	1.0
VP → VBD NP	0.4		

Sentence: The gunman sprayed the building with bullets.

Parse t_1: "with bullets" modifies the verb

```
[S [NP The gunman]
   [VP [VP sprayed [NP the building]]
      [PP with [NP bullets]]]]
```

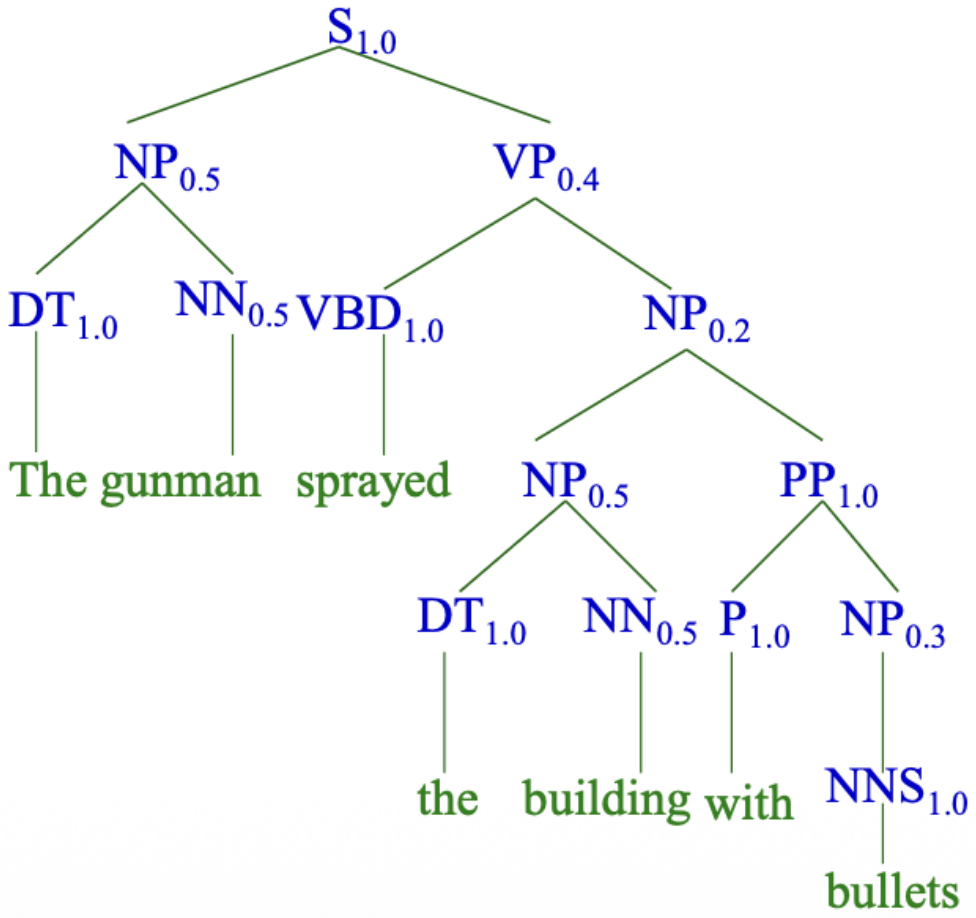


$$P(t_1) = 1.0 \times 0.5 \times 0.5 \times 0.6 \times 0.4 \times 0.5 \times 0.5 \times 1.0 \times$$
$$= 0.0045$$

Rules with probability 1.0 do not change the product.

Parse t_2: "with bullets" modifies the building

```
[S [NP The gunman]
   [VP sprayed
      [NP [NP the building]
          [PP with [NP bullets]]]]]
```



$P(t_2) = 1.0 \times 0.5 \times 0.5 \times 0.4 \times 0.2 \times 0.5 \times 0.5 \times 1.0 = 0.0015$

P(sentence) = 0.0045 + 0.0015 = 0.006. The PCFG prefers t_1.

Evaluating a constituency parser: PARSEVAL

	Gold: t_1 (VP attachment)	Predicted: t_2 (NP attachment)
shared	S[0,7], NP[0,2], VP[2,7], NP[3,5], PP[5,7], NP[6,7]	same six
different	VP[2,5]	NP[3,7]

Precision = 6/7 = 0.86

Recall = 6/7 = 0.86

F1 = 0.86

PART 5

Chart parsing: CKY and Earley

Dynamic programming over spans or chart states.

Parsing algorithms

CKY (Cocke–Kasami–Younger)

Bottom-up dynamic programming
Needs CFG grammars in Chomsky Normal Form
Builds constituents span by span

Earley

Top-down prediction + bottom-up completion
Works with any CFG
Processes input left to right

Chomsky Normal Form (CNF)

Allowed	Not allowed
A → B C	NP → the Nominal
NP → Nominal PP	S → VP
Noun → man	S → Aux NP VP

In CNF, every rule has either two non-terminals or one terminal on the right.

A small PCFG for English

Grammar rules

S → NP VP	.8
S → Aux NP VP	.1
S → VP	.1
NP → Pronoun	.2
NP → Proper-Noun	.2
NP → Det Nominal	.6
Nominal → Noun	.3
Nominal → Nominal Noun	.2
Nominal → Nominal PP	.5
VP → Verb	.2
VP → Verb NP	.5
VP → VP PP	.3
PP → Prep NP	1.0

Lexical rules (words)

Det → the | a | that | this
.6 .2 .1 .1

Noun → book | flight | meal | money
.1 .5 .2 .2

Verb → book | include | prefer
.5 .2 .3

Pronoun → I | he | she | me
.5 .1 .1 .3

Proper-Noun → Houston | NWA (.8, .2)
Aux → does 1.0
Prep → from | to | on | near | through
.25 .25 .1 .2 .2

For each left-hand side, the probabilities of its alternative expansions sum to 1.

Converting to CNF: examples

Original rule	CNF version
S → Aux NP VP	S → X1 VP, X1 → Aux NP
NP → the Nominal	NP → Det Nominal, Det → the
S → VP (and VP → Verb NP)	S → Verb NP
VP → Verb	VP → book | include | prefer

The key idea

If a constituent A spans words i to j, it is made of a B spanning i to k and a C spanning k to j, for some k between them.

$$\mathbf{A \rightarrow B\ C \text{ in } [i,j] \Leftarrow B \text{ in } [i,k] \text{ and } C \text{ in } [k,j]}$$

Store every constituent found for every span, and build longer spans from shorter ones. Nothing is computed twice.

The CKY table

- Positions are between words: (0) the (1) flight (2) includes (3) a (4) meal (5).
- Cell [i,j] holds every non-terminal that can span words i to j.
- Cells [j–1,j] hold pre-terminals (POS).
- If S is in cell [0,n], the sentence is recognized.

Example: grammar in CNF

```
S → NP VP
NP → Det N
VP → V NP
Det → the | a
N → flight | meal
V → includes
```

Sentence: the flight includes a meal (n = 5).

CKY step 1: the diagonal (lexical step)

	1	2	3	4	5
0	Det				
1		N			
2			V		
3				Det	
4					N

For each word, add every X with a rule X → word.

CKY step 2: spans of two words

	1	2	3	4	5
0	Det	NP			
1		N	–		
2			V	–	
3				Det	NP
4					N

[0,2]: Det + N → NP. Same for [3,5].

CKY: the finished table

	1	2	3	4	5
0	Det	NP	–	–	S
1		N	–	–	–
2			V	–	VP
3				Det	NP
4					N

[2,5]: V + NP → VP. [0,5]: NP + VP → S. Recognized.

The CKY algorithm

```
for j in 1..n:
    add {X | X → w_j} to table[j-1, j]                  # lexical
    for i in j-2 down to 0:
        for k in i+1 .. j-1:
            for each rule A → B C:
                if B in table[i,k] and C in table[k,j]:
                    add A to table[i,j]                 # syntactic
```

From recognition to parsing

- So far CKY only says yes or no.
- Store a backpointer with each entry: which rule, which split k, which two cells.
- Follow backpointers from S in [0,n] to rebuild the tree.
- S[0,5] → NP[0,2] VP[2,5]; VP[2,5] → V[2,3] NP[3,5].
- ⇒ [S [NP the flight] [VP includes [NP a meal]]]

CKY: cost

- Time $O(n^3 \cdot |G|)$: cubic in sentence length, linear in grammar size.
- Needs CNF; trees are converted back afterwards.
- Probabilistic CKY: each cell keeps the best probability per non-terminal, so the table yields the most probable parse.

Probabilistic CKY exercise

Rule	P	Rule	P
S → NP VP	1.0	NP → astronomers	0.2
NP → NP PP	0.4	NP → ears	0.18
PP → P NP	1.0	NP → saw	0.04
VP → V NP	0.7	NP → stars	0.18
VP → VP PP	0.3	V → saw	1.0
		P → with	1.0

Sentence: astronomers saw stars with ears

Probabilistic CKY: the update

Keep the best probability for each non-terminal in each cell:

$$P(A,[i,j]) = \max_{k,A\to BC} \; P(A\to BC) \times P(B,[i,k]) \times P(C,[k,j])$$

Example: $P(VP,[1,3]) = 0.7 \times 1.0 \times 0.18 = 0.126$.

The Earley algorithm: top-down with a chart

- One left-to-right pass; chart[k] holds every partial parse that ends at position k (n + 1 charts).
- A state is a dotted rule with a span [i,j]: the constituent starts at i; the dot marks progress.

0 Book 1 that 2 flight 3

```
S → • VP, [0,0]          predicted: a VP is expected at position 0
NP → Det • Nom, [1,2]    in progress: Det found, Nom expected next
VP → V NP •, [0,3]       completed: a VP covering the whole input
```

Success: a state S → α • [0,n] in the last chart.

Earley: three operators

Operator	When	What it adds (example)
Predictor	a non-terminal is **right** of the dot	S → •VP ⇒ VP → •V, VP → •V NP
Scanner	a POS is right of the dot and matches next **word**	VP → •V NP + book ⇒ VP → V •NP
Completer	the dot is at the **end** of a rule	NP → Det Nom • + VP → V •NP ⇒ VP → V NP •

Each state in chart[k] is processed once
New states go to chart[k] or chart[k+1]
No backtracking.

Earley trace: Book that flight

S → VP
VP → V NP
NP → Det Nom
Nom → N

V → book | include
Det → that | this
N → flight

Earley trace: Book that flight

Chart	State	Span	Added by
0	S → •VP	[0,0]	Predictor
0	VP → •V NP	[0,0]	Predictor
1	V → book •	[0,1]	Scanner
1	VP → V •NP	[0,1]	Completer
1	NP → •Det Nom	[1,1]	Predictor
2	Det → that •	[1,2]	Scanner
2	NP → Det •Nom	[1,2]	Completer
2	Nom → •N	[2,2]	Predictor
3	N → flight •	[2,3]	Scanner
3	NP → Det Nom •	[1,3]	Completer
3	S → VP •	[0,3]	Completer

CKY vs Earley

	CKY	Earley
Direction	bottom-up, span by span	left to right: top-down predictions, bottom-up completion
Grammar	CNF only	any CFG
Time	$O(n^3 \cdot \lvert G \rvert)$	$O(n^3)$ worst case; $O(n^2)$ for unambiguous grammars
Partial results	constituents for all spans	dotted rules: also what is still expected

Both use dynamic programming over a chart; both extend to probabilities.

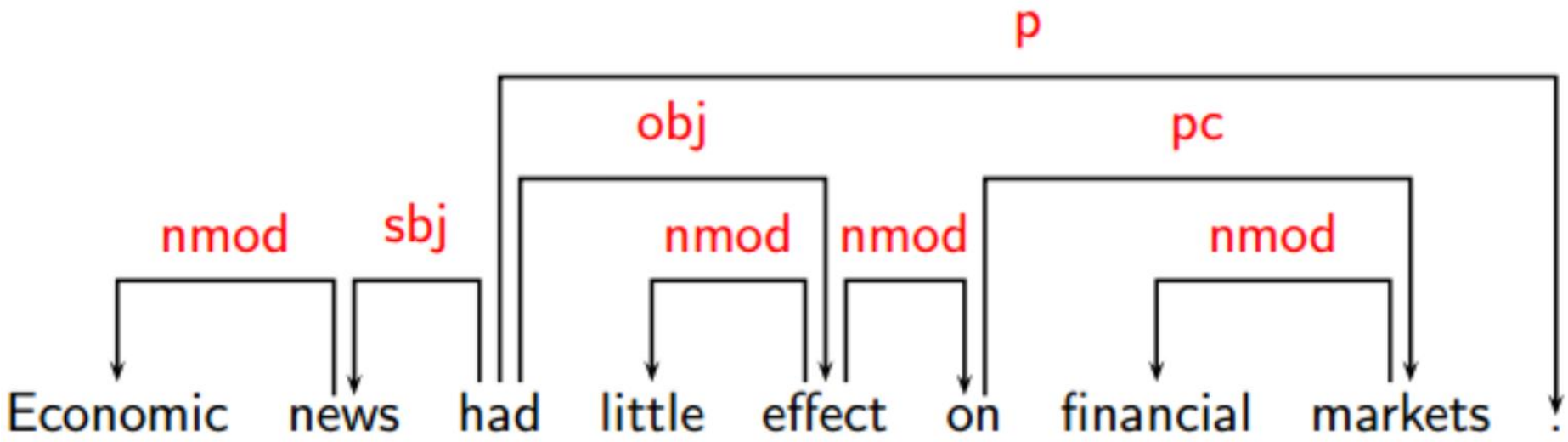
PART 6

Dependency parsing

Words linked directly to the words they depend on.

Dependency structure

- Structure = words linked by binary, asymmetric relations: head → dependent.
- No phrase nodes: a tree of relations between words.
- Relations usually carry labels (subject, object, modifier, ...).



Example: heads and dependents

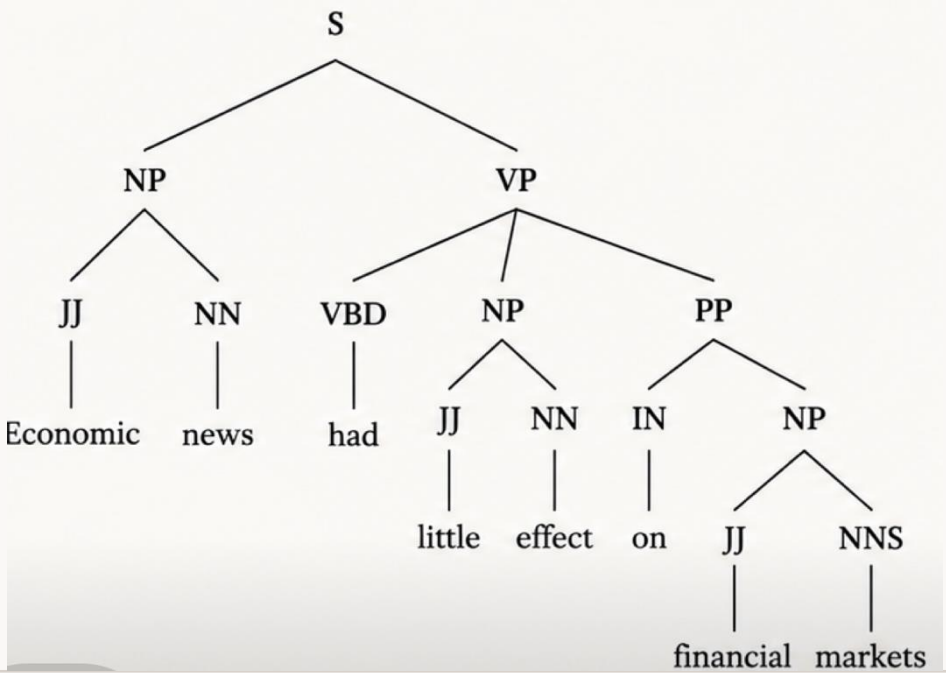
Dependent	Head	Relation
had	(root)	root
news	had	nsubj
Economic	news	amod
effect	had	obj
little	effect	amod
markets	effect	nmod
on	markets	case
financial	markets	amod

Economic news had little effect on financial markets.

Same sentence, both views

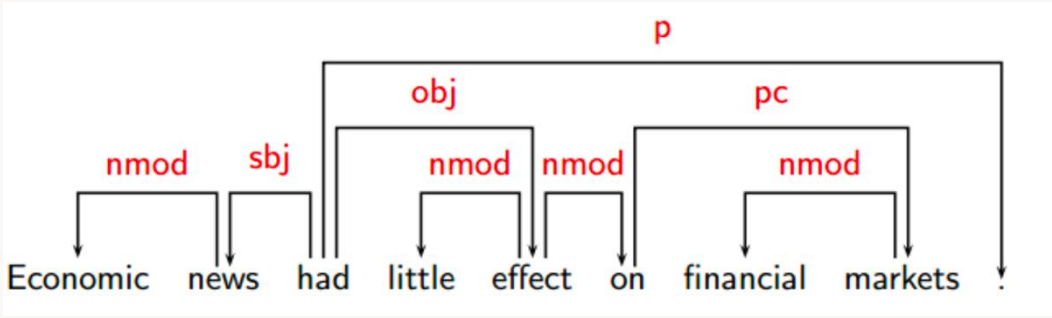
Constituency

[S [NP Economic news]
  [VP had
    [NP [NP little effect]
      [PP on [NP financial markets]]]]]



Dependency

had → news → Economic
had → effect → little
effect → markets → on
markets → financial



Universal Dependencies relations

Label	Meaning	Example (head → dependent)
nsubj	nominal subject	had → news
obj	direct object	had → effect
amod	adjectival modifier	news → Economic
det	determiner	flight → the
case	preposition / case marker	markets → on
nmod	nominal modifier	effect → markets

Universal Dependencies provides a shared inventory of relations across 100+ languages.

Dependencies and free word order

The commission proposed the regulation.
The regulation was proposed by the commission.

Different word order, same core relations:

propus → Commission (nsubj)
propus → regulation (obj)

This is why dependencies suit languages with freer word order, such as Czech or Hindi.

The CoNLL-U format: a real Romanian sentence

ID	FORM	LEMMA	UPOS	HEAD	DEPREL
1	Comisia	comisie	NOUN	2	nsubj
2	sesizează	sesiza	VERB	0	root
3	imediat	imediat	ADV	2	advmod
4	Comitetul	comitet	NOUN	2	obj
5	prevăzut	prevedea	VERB	4	acl
6	în	în	ADP	7	case
7	art.	articol	NOUN	5	obl
8	23	23	NUM	7	nummod
9	.	.	PUNCT	2	punct

UD Romanian-RRT, train-1696: “Comisia sesizează imediat Comitetul prevăzut în art. 23.”

What each representation makes explicit

Dependency	Phrase structure
head–dependent relations (arcs)	phrases (non-terminal nodes)
grammatical functions (arc labels)	phrase categories (NP, VP, ...)
sometimes POS	sometimes grammatical functions

Why dependencies became popular

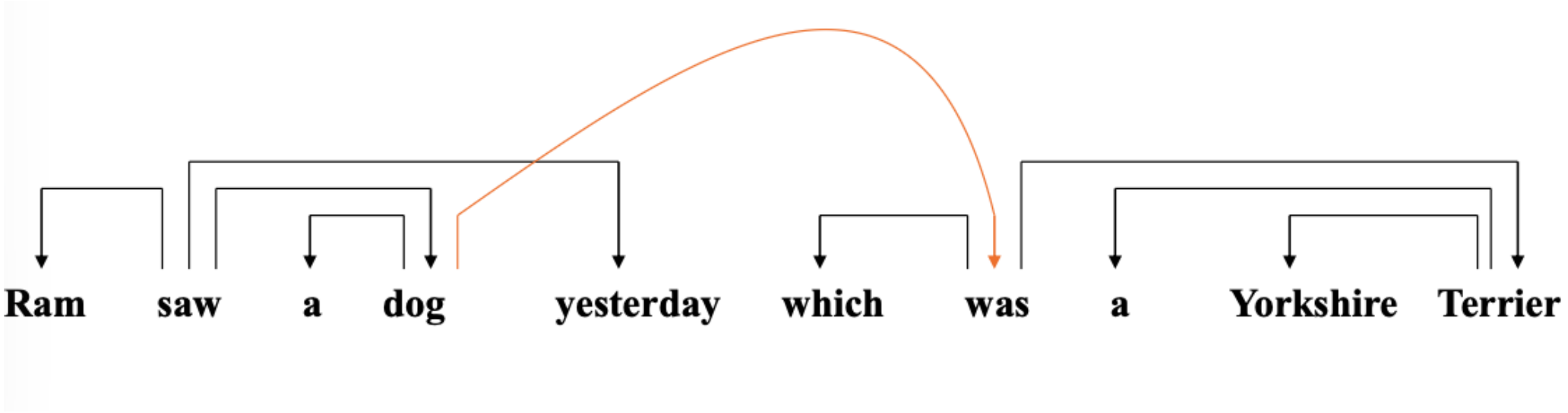
- Parsing reduces to choosing a head (and label) for each word.
- Predicate–argument structure is read off directly.
- Fragments are interpretable on their own.
- Less tied to word order.

Well-formed dependency trees

Condition	Violated if…
Single head	a word has two heads
Acyclic	A → B and B → A
Connected	some word is not attached
Projective	arcs don't cross when drawn above the sentence

Projectivity is a property of many, not all, dependency trees.

Non-projective trees: crossing arcs



English has a few cases of non-projective trees.

UD Romanian-RRT: 837 of 9,524 sentences (8.8%) have at least one crossing arc; yet only 1.2% of all arcs cross.

Dependency parsing approaches

Tradition	Examples
Dynamic programming	Eisner (1996); McDonald et al. (2005)
Constraint satisfaction	Maruyama (1990); Foth et al.
Transition-based	Yamada & Matsumoto (2003); Nivre (2003)
Neural	Chen & Manning (2014); Dozat & Manning (2017)

Grammar-driven and data-driven approaches can be combined.

Transition-based parsing: an example

Stack	Buffer	Action
[]	Economic news had …	SHIFT
[Economic]	news had …	SHIFT
[Economic news]	had …	LEFT-ARC amod
[news]	had …	SHIFT
[news had]	little effect …	LEFT-ARC nsubj
[had]	little effect …	SHIFT, SHIFT
[had little effect]	on financial markets	LEFT-ARC amod
[had effect]	on financial markets	SHIFT × 3
[… effect on financial markets]		LEFT-ARC amod, LEFT-ARC case
[had effect markets]		RIGHT-ARC nmod
[had effect]		RIGHT-ARC obj

Evaluating a dependency parser

Metric	Counts a word as correct if…
UAS	its head is right
LAS	its head and its label are right

UAS = Unlabeled Attachment Score
LAS = Labeled Attachment Score

Example: 9 words, 8 correct heads, 7 with the right label

UAS = 8/9 = 0.89

LAS = 7/9 = 0.78.

Parsers you can use

Tool	Notes
spaCy	fast dependency parsing for many languages
Stanza	UD models, Romanian included
UDPipe	trained on UD treebanks
Stanford CoreNLP	constituency and dependency

From classical to neural parsing

Classical

words + POS + handcrafted features
→ parser
→ tree

Neural

tokens
→ contextual encoder
→ arc / span scores
→ best tree

From syntax to meaning: semantic roles

[The Commission] proposed [the AI Act]
[The AI Act] was proposed [by the Commission]

Syntax changes

The subject is first the Commission, then the Act.

Roles stay

Agent: the Commission
Theme: the AI Act

Syntactic structure can support semantic role labelling, but modern SRL can also work directly from contextual representations.

Resources: PropBank, FrameNet.

Summary

	Constituency	Dependency
Structure	nested phrases (NP, VP, PP)	head → dependent arcs, labelled
Model	(P)CFG: tree probability = product of rule probabilities	scores for arcs or transitions
Algorithms	CKY (CNF) and Earley (any CFG)	transition-based, graph-based, neural
Evaluation	PARSEVAL precision / recall / F1	UAS, LAS

Ambiguity is the central problem: probabilities or neural scores choose among possible structures.

References

- Jurafsky & Martin (2026). Speech and Language Processing, 3rd ed., online manuscript, Aug. 19, 2026.
- Earley (1970). An efficient context-free parsing algorithm. Communications of the ACM 13(2): 94–102.
- Manning & Schütze (1999). Foundations of Statistical Natural Language Processing. MIT Press.
- Shiber (1985). Evidence against the context-freeness of natural language. Linguistics and Philosophy 8.
- Nivre (2008). Algorithms for deterministic incremental dependency parsing. Computational Linguistics 34(4).
- Chen & Manning (2014). A Fast and Accurate Dependency Parser using Neural Networks. EMNLP.
- Dozat & Manning (2017). Deep Biaffine Attention for Neural Dependency Parsing. ICLR.
- de Marneffe, Manning, Nivre & Zeman (2021). Universal Dependencies. Computational Linguistics 47(2): 255–308.

Thank you!

diana.trandabat@info.uaic.ro

PART A

Appendix

Formal-language details, a full probabilistic CKY exercise and lab material.

Type 3: regular grammars

Right-linear

$A \rightarrow xB$
$A \rightarrow x$

Left-linear

$A \rightarrow Bx$
$A \rightarrow x$

A regular grammar for (tiny) English

```
S → the A
A → cat B | mouse B | duck B
B → bites C | sees C | eats C
C → the D
D → boy | girl | monkey
```

Generates, for example:

the cat bites the boy
the mouse eats the monkey
the duck sees the girl

A regular rule set for noun phrases

```
NP → article NP1
NP1 → adjective NP1
NP1 → noun

NP ⇒ the NP1
   ⇒ the old NP1
   ⇒ the old tall NP1
   ⇒ the old tall man
```

Type 2: context-free grammars

Rules have one non-terminal on the left and any sequence on the right:

$$A \rightarrow \gamma$$

Examples: S → NP VP, NP → Det N, NP → NP PP.
Recursion, including centre embedding (A ⇒ … A …), is allowed.

Appendix: beyond context-free, Swiss German

… mer em Hans es huus hälfed aastriiche
… we Hans.DAT the house.ACC helped paint
"… we helped Hans paint the house"

Dative nouns, accusative nouns, dative verbs, accusative verbs: $a^n b^m c^n d^m$.
These cross-serial dependencies are beyond context-free grammars.

Grammar symbols

Symbol type	Definition	In the example
Non-terminal	appears on a left-hand side	S, NP, VP, Det, N
Terminal	appears only in the final string	the, dog
Pre-terminal	non-terminal that rewrites to a word	Det, N (the POS tags)

Sequences of pre-terminals show the shape of a sentence: Det N V Det N.

Which grammar is best?

- The structure a grammar assigns should be understandable.
- It should allow useful generalizations (e.g. VP can be coordinated: John kicks Bill and runs).
- It should reflect the meaning of the sentence.

Lab exercise: which grammars are regular?

```
G1: A → a | aB
    B → bA

G2: A → a | Ba
    B → Ab

G3: A → a | aB
    B → b | bA

G4: A → a | Ba
    B → b | Ab

G5: S → AB | ε
    A → aS
    B → bbS

G6: S → bB
    A → a | Ba
    B → b | bA
```

Lab exercise: answers

Grammar	Regular?	Reason
G1, G3	yes	all rules right-linear
G2, G4	yes	all rules left-linear
G5	no	S → AB has two non-terminals
G6	no	mixes right- and left-linear rules

A grammar that is not regular may still describe a regular language; the question here is about the rules.

Practice: the filled table

	1	2	3	4	5
0	NP .2	–	S .0252	–	S .0018144
1		NP .04, V 1	VP .126	–	VP .009072
2			NP .18	–	NP .01296
3				P 1	PP .18
4					NP .18

Practice: two parses

Parse	Probability
[saw [stars with ears]] (NP attachment)	0.0018144
[[saw stars] [with ears]] (VP attachment)	0.0013608
P(sentence) = sum	0.0031752

Probabilistic CKY keeps the first parse: stars that have ears.

CKY with unary rules

- Grammars often contain unary rules such as NP → N or S → VP.
- After the lexical step, apply unary rules in each diagonal cell.
- In every other cell, after the binary rules, apply unary rules to the new entries.
- Example: N in [1,2] + NP → N ⇒ NP also in [1,2].

Appendix: Earley exercise

Parse “Papa ate the caviar with a spoon” with Earley’s algorithm. How many parses are there?

```
S → NP VP          NP → Papa
NP → Det N         N → caviar | spoon
NP → NP PP         V → ate
VP → VP PP         P → with
VP → V NP          Det → the | a
PP → P NP
```